

INTERACTIVE LEARNING

SQL Server

Mastering Queries

Topics: Dates • WHERE • Aggregates • HAVING • ORDER BY • GROUP BY • UNION

SQL Server Edition

What We'll Cover Today

01

DATE Functions

GETDATE, DATEPART, DATEDIFF, DATEADD

02

WHERE Clause

Filtering rows with conditions

03

Aggregate Functions

SUM, COUNT, AVG, MIN, MAX

04

ORDER BY

Sorting query results

05

GROUP BY

Grouping rows for aggregation

06

WHERE vs HAVING

Key differences explained

07

HAVING Clause

Filtering grouped results

08

UNION & UNION ALL

Combining result sets

SQL SERVER

DATE Functions

Working with dates and times in SQL Server

DATE Functions – GETDATE() & DATEPART()

GETDATE()

Returns the current date and time of the SQL Server.

```
SELECT GETDATE() AS CurrentDateTime;  
  
-- Output (example):  
-- 2024-04-24 10:35:22.123
```

DATEPART()

Extracts a specific part (year, month, day...) from a date.

```
SELECT DATEPART(YEAR, GETDATE()) AS Yr,  
       DATEPART(MONTH, GETDATE()) AS Mo,  
       DATEPART(DAY, GETDATE()) AS Dy;
```

Common Date Parts

Part	Meaning	Example Value
YEAR	Year number	2024
MONTH	Month number	4
DAY	Day of month	24
HOUR	Hour (0-23)	10
MINUTE	Minute	35
WEEKDAY	Day of week	4

💡 Use `SYSDATETIME()` when you need higher precision (nanoseconds).

DATE Functions – DATEDIFF & DATEADD

DATEDIFF()

Calculates the difference between two dates.

```
-- Syntax: DATEDIFF(part, start_date, end_date)

SELECT DATEDIFF(DAY, '2024-01-01', GETDATE())
       AS DaysSinceNewYear;

SELECT DATEDIFF(YEAR, BirthDate, GETDATE())
       AS Age
FROM   Employees;
```

DATEADD()

Adds or subtracts a time interval from a date.

```
-- Syntax: DATEADD(part, number, date)

SELECT DATEADD(DAY, 30, GETDATE()) AS In30Days;
SELECT DATEADD(MONTH, -3, GETDATE()) AS
ThreeMonthsAgo;
SELECT DATEADD(YEAR, 1, '2024-01-01') AS NextYear;
```



Practice Query

```
-- Find employees hired in last 6 months

SELECT EmpName, HireDate,
       DATEDIFF(DAY, HireDate, GETDATE())
       AS DaysEmployed
FROM   Employees
WHERE  HireDate >= DATEADD(MONTH, -6, GETDATE())
ORDER BY HireDate DESC;
```

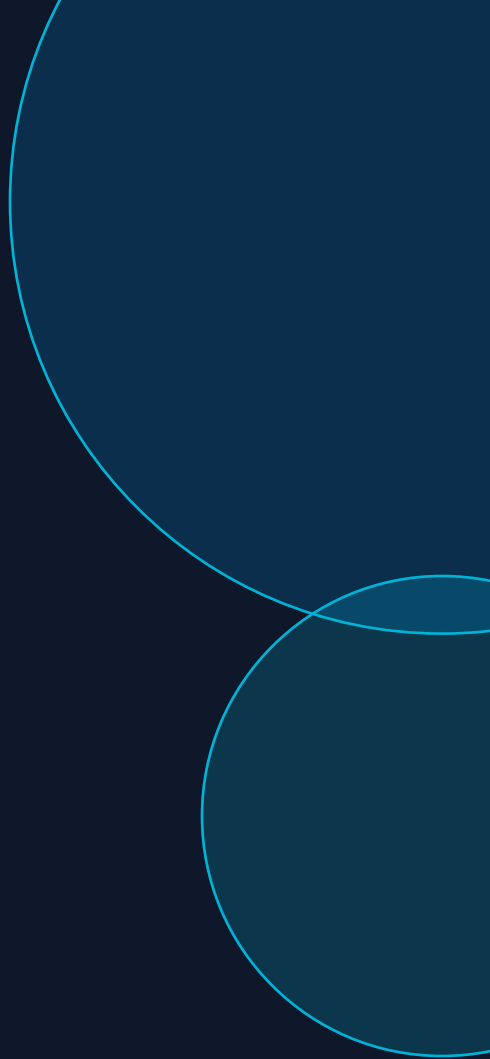
EmpName	HireDate	DaysEmployed
Rahul	2024-02-10	74
Priya	2024-01-05	110
Arjun	2023-11-20	156



Negative numbers in DATEADD go backward in time.

WHERE Clause

Filtering rows based on conditions



WHERE Clause – Filtering Data

Syntax

```
SELECT column1, column2
FROM   TableName
WHERE  condition;
```

Common Operators

Operator	Meaning	Example
=	Equal to	Salary = 50000
> / <	Greater / Less	Age > 30
>= / <=	Greater-or-equal	Marks >= 60
<> or !=	Not equal	Dept <> 'HR'
BETWEEN	In a range	Salary BETWEEN 30K AND 60K
LIKE	Pattern match	Name LIKE 'A%'
IN	List of values	City IN ('Surat','Mumbai')
IS NULL	Null check	Email IS NULL

Examples

```
-- Single condition
SELECT * FROM Employees
WHERE  Department = 'Sales';

-- AND / OR
SELECT * FROM Employees
WHERE  Salary > 40000 AND City = 'Surat';

-- BETWEEN
SELECT * FROM Orders
WHERE  OrderDate BETWEEN '2024-01-01'
                        AND '2024-12-31';

-- LIKE (starts with R)
SELECT * FROM Employees
WHERE  Name LIKE 'R%';

-- IN list
SELECT * FROM Products
WHERE  Category IN ('Electronics', 'Clothing');
```



Use AND to narrow results, OR to expand. Wrap OR groups in parentheses!

SQL SERVER

Aggregate Functions

SUM · COUNT · AVG · MIN · MAX

Aggregate Functions

SUM()

Total of all values

```
SELECT SUM(Salary) AS TotalSalary
FROM   Employees;
```

COUNT()

Number of rows / values

```
SELECT COUNT(*) AS TotalEmp
FROM   Employees;
SELECT COUNT(Email) AS WithEmail
FROM   Employees; -- ignores NULLs
```

AVG()

Average of values

```
SELECT AVG(Salary) AS AvgSalary
FROM   Employees
WHERE  Department = 'IT';
```

MIN() / MAX()

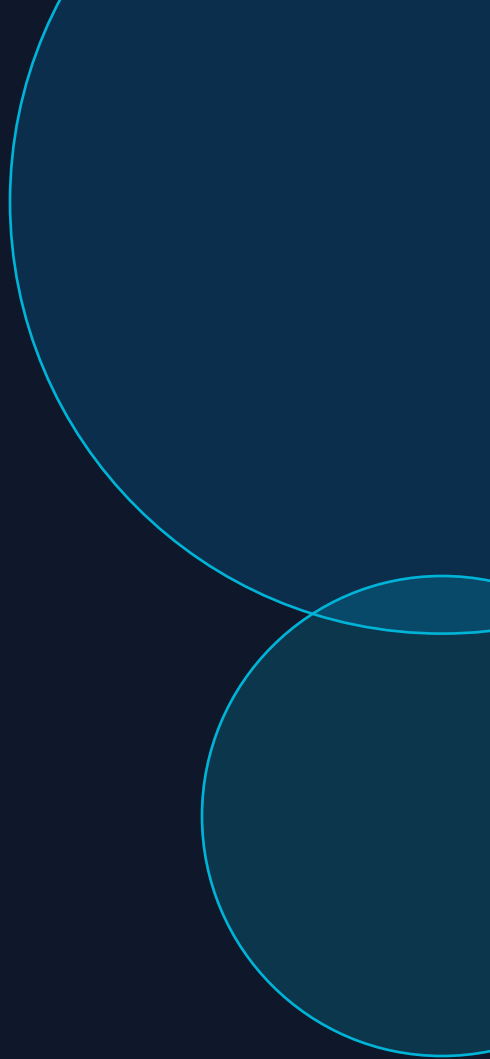
Smallest / Largest value

```
SELECT MIN(Salary) AS MinSal,
       MAX(Salary) AS MaxSal
FROM   Employees;
```

SQL SERVER

ORDER BY

Sorting your query results



ORDER BY Clause

Sorts the result set. Default is ASC (smallest first). Use DESC for largest first.

```
-- Ascending (default)
SELECT EmpName, Salary
FROM Employees
ORDER BY Salary;           -- lowest first (ASC)

-- Descending
SELECT EmpName, Salary
FROM Employees
ORDER BY Salary DESC;      -- highest first

-- Multiple columns: first by dept ASC, then salary DESC
SELECT EmpName, Department, Salary
FROM Employees
ORDER BY Department ASC, Salary DESC;

-- Order by column position (column number in SELECT list)
SELECT EmpName, Department, Salary
FROM Employees
ORDER BY 3 DESC;           -- 3rd column = Salary

-- Order by alias
SELECT EmpName,
       Salary * 12 AS AnnualPay
FROM Employees
ORDER BY AnnualPay DESC;
```

Result Sample

EmpName	Dept	Salary
Priya	Finance	75000
Rahul	IT	65000
Arjun	IT	60000
Neha	HR	55000
Kiran	Sales	45000

Important Notes

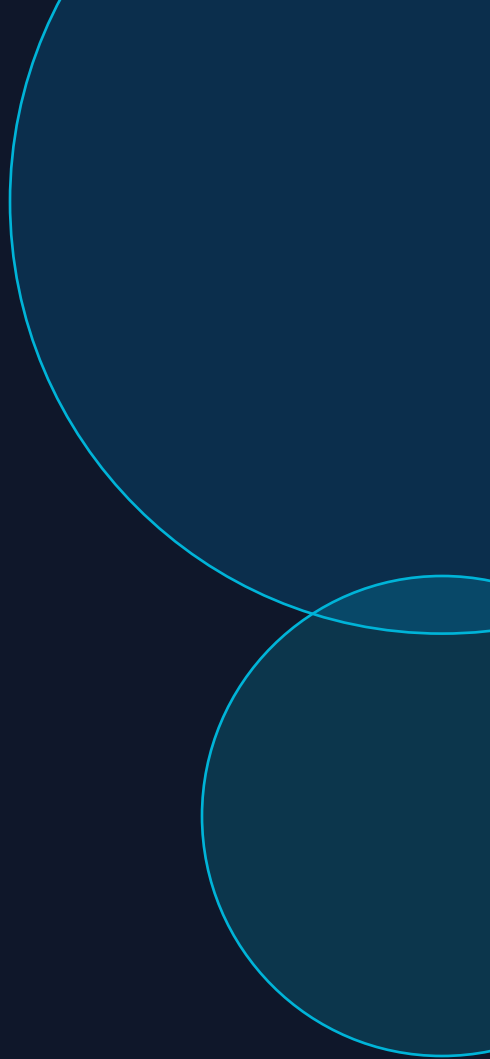
- ORDER BY is always the LAST clause
- NULLs appear first in ASC order
- Use TOP with ORDER BY to get top N rows

```
-- TOP 3 highest paid employees
SELECT TOP 3 EmpName, Salary
FROM Employees
```

SQL SERVER

GROUP BY

Grouping rows to apply aggregate functions



GROUP BY Clause

What it does:

Groups rows that have the same values in specified columns, so aggregate functions can be applied to each group.

```
-- Total salary per department
SELECT  Department,
        COUNT(*)      AS NumEmployees,
        SUM(Salary)   AS TotalSalary,
        AVG(Salary)   AS AvgSalary
FROM    Employees
GROUP BY Department;
```

⚠ Rule: Every column in SELECT that is NOT inside an aggregate function MUST appear in GROUP BY.

```
-- Group by multiple columns
SELECT  Department, City,
        COUNT(*) AS Headcount
FROM    Employees
GROUP BY Department, City
ORDER BY Department;
```

Result Preview

Department	NumEmp	TotalSal	AvgSal
IT	8	480000	60000
HR	5	250000	50000
Sales	12	600000	50000
Finance	6	360000	60000

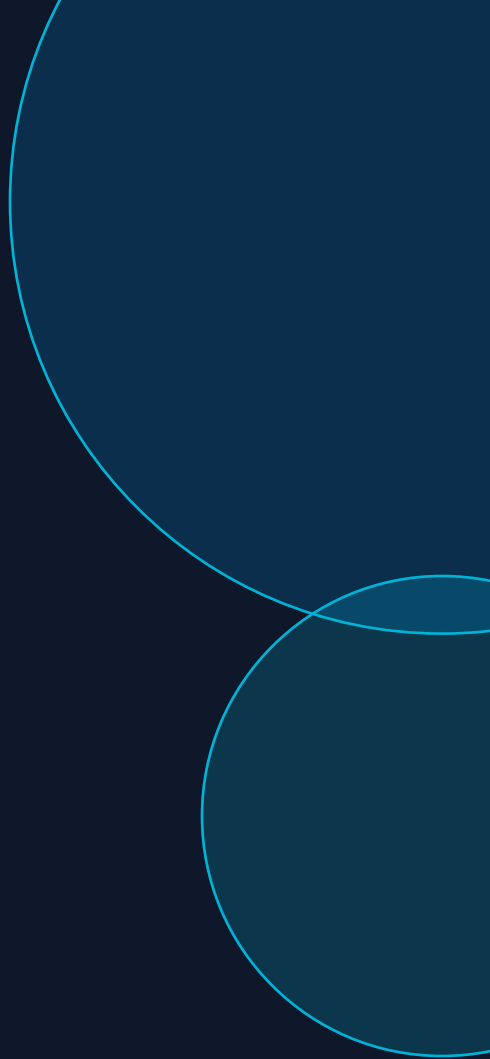
Execution Order

- ① FROM – get the table
- ② WHERE – filter rows
- ③ GROUP BY – form groups
- ④ HAVING – filter groups
- ⑤ SELECT – compute output
- ⑥ ORDER BY – sort

SQL SERVER

HAVING Clause

Filtering groups after aggregation



HAVING filters groups created by GROUP BY — just like WHERE filters rows. **Result**

```
-- Show only departments with more than 5 employees
SELECT  Department,
        COUNT(*) AS NumEmployees,
        AVG(Salary) AS AvgSalary
FROM    Employees
GROUP BY Department
HAVING  COUNT(*) > 5;
```

```
-- Departments with total salary > 300,000
SELECT  Department, SUM(Salary) AS TotalSal
FROM    Employees
GROUP BY Department
HAVING  SUM(Salary) > 300000
ORDER BY TotalSal DESC;
```

Department	NumEmp	AvgSalary
IT	8	60000
Sales	12	50000
Finance	6	60000

Key Points

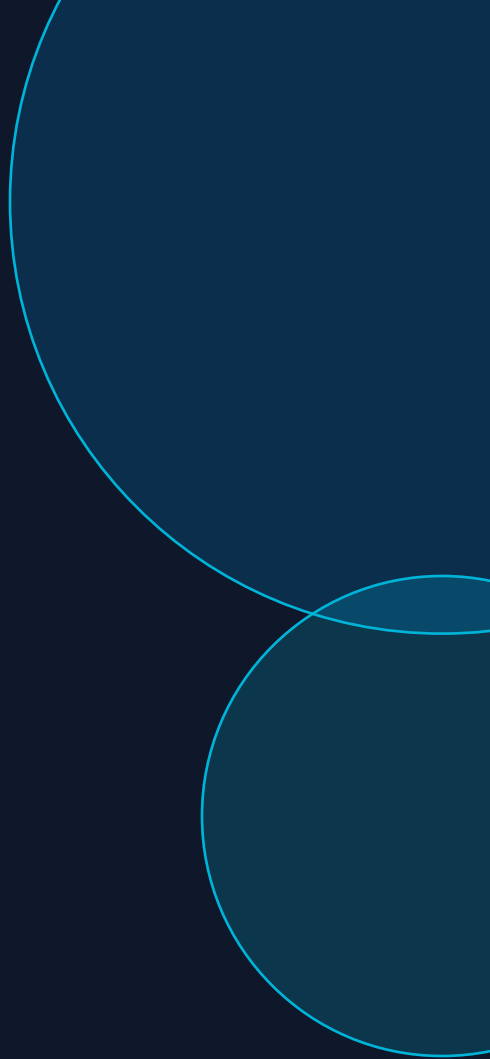
- HAVING works on group-level data
- Can use aggregate functions (SUM, COUNT, AVG...)
- Always comes AFTER GROUP BY
- Cannot reference aliases defined in SELECT

```
-- Combine WHERE + HAVING
SELECT  Department, COUNT(*) AS Cnt
FROM    Employees
WHERE   Salary > 30000      -- filters rows first
GROUP BY Department
HAVING  COUNT(*) >= 3;     -- then filters groups
```

SQL SERVER

WHERE vs HAVING

Understanding the key differences



WHERE vs HAVING – Side by Side

Feature	WHERE	HAVING
Filters	Individual rows	Groups of rows
Used with	SELECT, UPDATE, DELETE	Only SELECT with GROUP BY
Position in query	Before GROUP BY	After GROUP BY
Aggregates allowed?	✗ No	✓ Yes
Applied	Before grouping	After grouping
Example condition	Salary > 50000	COUNT(*) > 5

```
-- WHERE filters before grouping (no aggregates)
SELECT Department, COUNT(*) AS Cnt
FROM Employees
WHERE Salary > 30000          -- ✓ filters individual rows
GROUP BY Department;
```

```
-- HAVING filters after grouping (uses aggregates)
SELECT Department, COUNT(*) AS Cnt
FROM Employees
GROUP BY Department
HAVING COUNT(*) > 5;          -- ✓ filters groups
```

```
-- WRONG: Cannot use aggregate in WHERE
SELECT Department
FROM Employees
WHERE COUNT(*) > 5;          -- ✗ ERROR!
```

💡 Simple rule: If you need to filter using an aggregate (SUM, COUNT, AVG...) → use HAVING. For everything else → use WHERE.

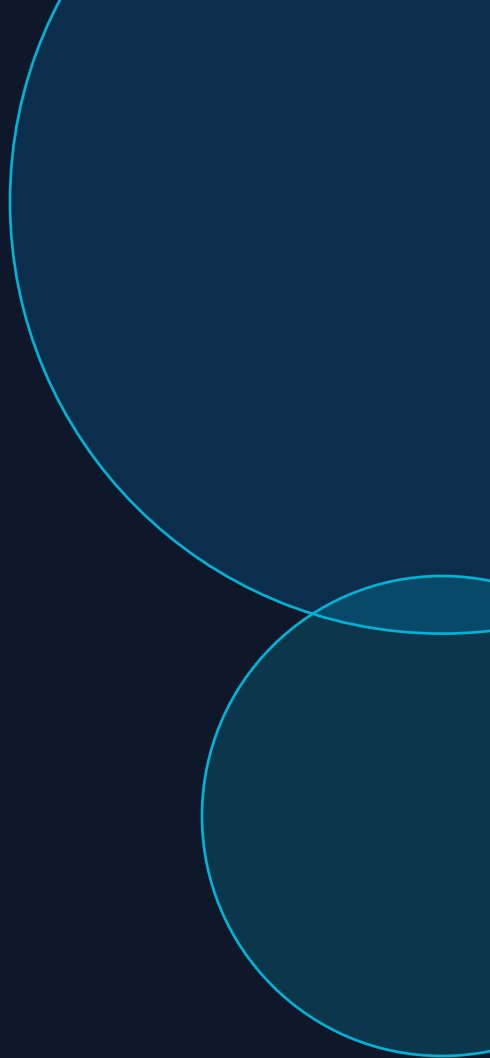
Quick Memory Trick

WHERE → rows | HAVING → groups

SQL SERVER

UNION & UNION ALL

Combining results from multiple queries



UNION vs UNION ALL

UNION

Combines results and removes DUPLICATES.

```
SELECT City FROM Customers
UNION
SELECT City FROM Suppliers;
-- Returns unique cities only
```

Performance: Slower (removes duplicates using sort/hash)

City
Surat
Mumbai
Delhi
Ahmedabad

← Duplicates removed!

UNION ALL

Combines results and KEEPS all duplicates.

```
SELECT City FROM Customers
UNION ALL
SELECT City FROM Suppliers;
-- Returns all rows including dupes
```

Performance: Faster (no duplicate removal needed)

City
Surat
Mumbai
Surat
Delhi
Mumbai
Ahmedabad

← Duplicates kept!

UNION – Rules & Practice

Rules for UNION / UNION ALL

- Same number of columns in both queries
- Columns must have compatible data types
- Column names come from the FIRST query
- ORDER BY goes at the very end only
- Can combine more than two queries

```
-- Combine current + former employees
SELECT EmpName, 'Active' AS Status FROM Employees
UNION ALL
SELECT EmpName, 'Former' AS Status FROM FormerEmployees
ORDER BY EmpName;
```

```
-- Wrong: different column counts ❌
SELECT Name, Salary FROM Employees
UNION
SELECT Name FROM Customers;      -- ❌ ERROR
```

```
-- Correct fix ✅
SELECT Name, Salary FROM Employees
UNION
SELECT Name, 0 FROM Customers; -- pad with 0
```

When to use which?

Use UNION when...

You want distinct/unique results only.
E.g. unique list of cities across two tables.

Use UNION ALL when...

You want ALL rows including duplicates, or when you know there are no duplicates (faster).
E.g. combining transaction logs.



When in doubt: use UNION ALL first for speed, then add UNION if you need to deduplicate.

SQL Cheat Sheet – Quick Reference

Clause / Function	Purpose	Key Rule
GETDATE()	Current date & time	Returns datetime of SQL Server
DATEDIFF(part,s,e)	Difference between dates	part = YEAR/MONTH/DAY etc.
DATEADD(part,n,d)	Add/subtract from date	Use negative n to go back
WHERE	Filter rows	Cannot use aggregate functions
SUM / COUNT / AVG	Aggregate values	Ignores NULLs (except COUNT(*))
GROUP BY	Form groups for aggregates	All non-aggregate cols must be here
HAVING	Filter groups	Used after GROUP BY; allows aggregates
ORDER BY	Sort results	Last clause; ASC default
UNION	Combine queries (unique)	Same cols, compatible types; slow
UNION ALL	Combine queries (all rows)	Same cols; faster; keeps duplicates



Query Execution Order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

Thank You!

Keep Practicing SQL!

Practice daily • Use SQL Server Management Studio (SSMS) • Build real projects

Topics Covered: Dates • WHERE • Aggregates • GROUP BY • HAVING • ORDER BY • UNION